TITLE: Smart Waste Collection and Recycling Management System

PSEUDOCODE

START

Create a dictionary named waste_data

Store Plastic, Paper, Glass, Metal and Organic as categories

Create a set containing recyclable waste categories

Load previously saved records from the file

REPEAT

Display the main menu

1. Add Waste Collection Record

   Get resident name

   Get waste category

   Get quantity

   Get collection date

   Validate the entered details

   Store the record as a tuple

   Add the tuple to the appropriate list

   Display success message

2. Display Records

   For each waste category

      Display all records stored in its list

   End For

3. Search Record

Get resident name

Search the name in all category lists

If record is found

Display the record

Else

Display "Record not found"

End If

4. Update Record

Get resident name

Search for the resident

If record is found

Get new quantity

Update the record

Else

Display "Record not found"

End If

5. Classify Waste

Get waste category

Check the category

If category belongs to recyclable set

Display "Recyclable Waste"

Else

Display "Organic Waste"

End If

6. Calculate Recyclable Quantity

Set total to 0

For each recyclable category

Add the quantity of all records

End For

Display total recyclable quantity

7. Category-wise Summary

For each waste category

Calculate total quantity

Display category and total quantity

End For

8. Calculate Total Waste

Set total to 0

Add quantities from all categories

Display total waste collected

9. Save Records

Open the file

Write all waste records into the file

Close the file

Display success message

10. Exit

Save all records

Close the application

UNTIL user selects Exit

END

SOURCE CODE

```
import tkinter as tk
from tkinter import ttk, messagebox
import os

waste_data = {
    "Plastic": [],
    "Paper": [],
    "Glass": [],
    "Metal": [],
    "Organic": []
}

recyclable = {"Plastic", "Paper", "Glass", "Metal"}
file_name = "waste_records.txt"



def add_record():
    name = name_entry.get().strip()
    category = category_box.get()
    quantity = quantity_entry.get().strip()
    date = date_entry.get().strip()

    if name == "" or quantity == "" or date == "":
        messagebox.showwarning("Warning", "Please enter all details.")
        return

    try:
        quantity = float(quantity)

        if quantity <= 0:
```

```
            messagebox.showwarning(
                "Warning",
                "Quantity must be greater than 0."
            )
            return

    except ValueError:
        messagebox.showerror(
            "Error",
            "Please enter a valid quantity."
        )
        return

    record = (name, quantity, date)
    waste_data[category].append(record)

    messagebox.showinfo(
        "Success",
        "Waste record added successfully."
    )

    clear_boxes()
    show_records()


def show_records():
    output.delete("1.0", tk.END)

    output.insert(
        tk.END,
        "========== WASTE COLLECTION RECORDS ==========\n\n"
```

```
)

found = False

for category in waste_data:
    records = waste_data[category]

    if len(records) > 0:
        found = True

        output.insert(
            tk.END,
            category.upper() + "\n"
        )

        for record in records:
            name = record[0]
            quantity = record[1]
            date = record[2]

            output.insert(
                tk.END,
                "Name: " + name +
                " | Quantity: " + str(quantity) +
                " kg | Date: " + date + "\n"
            )

        output.insert(tk.END, "\n")

if found == False:
    output.insert(tk.END, "No records available.")
```

```
def search_record():
    search_name = name_entry.get().strip().lower()

    if search_name == "":
        messagebox.showwarning(
            "Warning",
            "Enter a resident name first."
        )
        return

    output.delete("1.0", tk.END)

    found = False

    output.insert(
        tk.END,
        "========== SEARCH RESULT ==========\n\n"
    )

    for category in waste_data:
        for record in waste_data[category]:

            name = record[0]
            quantity = record[1]
            date = record[2]

            if search_name in name.lower():

                output.insert(
```

```
                tk.END,
                "Name: " + name + "\n"
            )

            output.insert(
                tk.END,
                "Category: " + category + "\n"
            )

            output.insert(
                tk.END,
                "Quantity: " + str(quantity) + " kg\n"
            )

            output.insert(
                tk.END,
                "Date: " + date + "\n\n"
            )

            found = True

    if found == False:
        output.insert(
            tk.END,
            "No record found."
        )


def update_record():
    search_name = name_entry.get().strip().lower()
    new_quantity = quantity_entry.get().strip()
```

```
if search_name == "" or new_quantity == "":
    messagebox.showwarning(
        "Warning",
        "Enter name and new quantity."
    )
    return

try:
    new_quantity = float(new_quantity)

    if new_quantity <= 0:
        messagebox.showwarning(
            "Warning",
            "Quantity must be greater than 0."
        )
        return

except ValueError:
    messagebox.showerror(
        "Error",
        "Enter a valid quantity."
    )
    return

for category in waste_data:
    records = waste_data[category]

    for i in range(len(records)):
        name = records[i][0]
        date = records[i][2]
```

```
            if name.lower() == search_name:

                records[i] = (
                    name,
                    new_quantity,
                    date
                )

                messagebox.showinfo(
                    "Success",
                    "Record updated successfully."
                )

                show_records()
                return

    messagebox.showwarning(
        "Not Found",
        "Record not found."
    )


def classify_waste():
    category = category_box.get()

    if category in recyclable:
        messagebox.showinfo(
            "Classification",
            category + " is recyclable waste."
        )
```

```
    else:
        messagebox.showinfo(
            "Classification",
            category + " is organic waste."
        )


def recyclable_quantity():
    total = 0

    for category in recyclable:
        for record in waste_data[category]:
            total = total + record[1]

    output.delete("1.0", tk.END)

    output.insert(
        tk.END,
        "========== RECYCLABLE WASTE ==========\n\n"
    )

    output.insert(
        tk.END,
        "Total recyclable waste: " +
        str(total) +
        " kg"
    )


def category_summary():
    output.delete("1.0", tk.END)
```

```
    output.insert(
        tk.END,
        "========== CATEGORY SUMMARY ==========\n\n"
    )

    for category in waste_data:

        total = 0

        for record in waste_data[category]:
            total = total + record[1]

        output.insert(
            tk.END,
            category + " : " +
            str(total) +
            " kg\n"
        )


def total_waste():
    total = 0

    for category in waste_data:
        for record in waste_data[category]:
            total = total + record[1]

    output.delete("1.0", tk.END)

    output.insert(
```

```
        tk.END,
        "========== TOTAL WASTE ==========\n\n"
    )

    output.insert(
        tk.END,
        "Total waste collected: " +
        str(total) +
        " kg"
    )


def save_records():
    file = open(file_name, "w")

    for category in waste_data:
        for record in waste_data[category]:

            name = record[0]
            quantity = record[1]
            date = record[2]

            file.write(
                category + "|" +
                name + "|" +
                str(quantity) + "|" +
                date + "\n"
            )

    file.close()
```

```
    messagebox.showinfo(
        "Saved",
        "Records saved successfully."
    )


def load_records():

    if os.path.exists(file_name) == False:
        return

    file = open(file_name, "r")

    for line in file:

        data = line.strip().split("|")

        if len(data) == 4:

            category = data[0]
            name = data[1]

            try:
                quantity = float(data[2])
            except ValueError:
                continue

            date = data[3]

            if category in waste_data:
```

```
            record = (
                name,
                quantity,
                date
            )

            waste_data[category].append(record)

    file.close()


def clear_boxes():
    name_entry.delete(0, tk.END)
    quantity_entry.delete(0, tk.END)
    date_entry.delete(0, tk.END)
    category_box.set("Plastic")


def exit_program():
    save_records()
    window.destroy()


window = tk.Tk()
window.title("Smart Waste Management")
window.geometry("850x650")
window.configure(bg="#e8f5e9")

title = tk.Label(
    window,
    text="SMART WASTE COLLECTION SYSTEM",
```

```
    font=("Arial", 20, "bold"),
    bg="#2e7d32",
    fg="white"
)

title.pack(fill="x", pady=10)

input_frame = tk.Frame(
    window,
    bg="#e8f5e9"
)

input_frame.pack(pady=10)

tk.Label(
    input_frame,
    text="Resident Name:",
    font=("Arial", 12),
    bg="#e8f5e9"
).grid(row=0, column=0, padx=10, pady=8)

name_entry = tk.Entry(
    input_frame,
    width=30
)

name_entry.grid(
    row=0,
    column=1,
    padx=10,
    pady=8
```

```
)

tk.Label(
    input_frame,
    text="Waste Category:",
    font=("Arial", 12),
    bg="#e8f5e9"
).grid(row=1, column=0, padx=10, pady=8)

category_box = ttk.Combobox(
    input_frame,
    values=[
        "Plastic",
        "Paper",
        "Glass",
        "Metal",
        "Organic"
    ],
    width=27,
    state="readonly"
)

category_box.set("Plastic")

category_box.grid(
    row=1,
    column=1,
    padx=10,
    pady=8
)
```

```
tk.Label(
    input_frame,
    text="Quantity (kg):",
    font=("Arial", 12),
    bg="#e8f5e9"
).grid(row=2, column=0, padx=10, pady=8)

quantity_entry = tk.Entry(
    input_frame,
    width=30
)

quantity_entry.grid(
    row=2,
    column=1,
    padx=10,
    pady=8
)

tk.Label(
    input_frame,
    text="Collection Date:",
    font=("Arial", 12),
    bg="#e8f5e9"
).grid(row=3, column=0, padx=10, pady=8)

date_entry = tk.Entry(
    input_frame,
    width=30
)
```

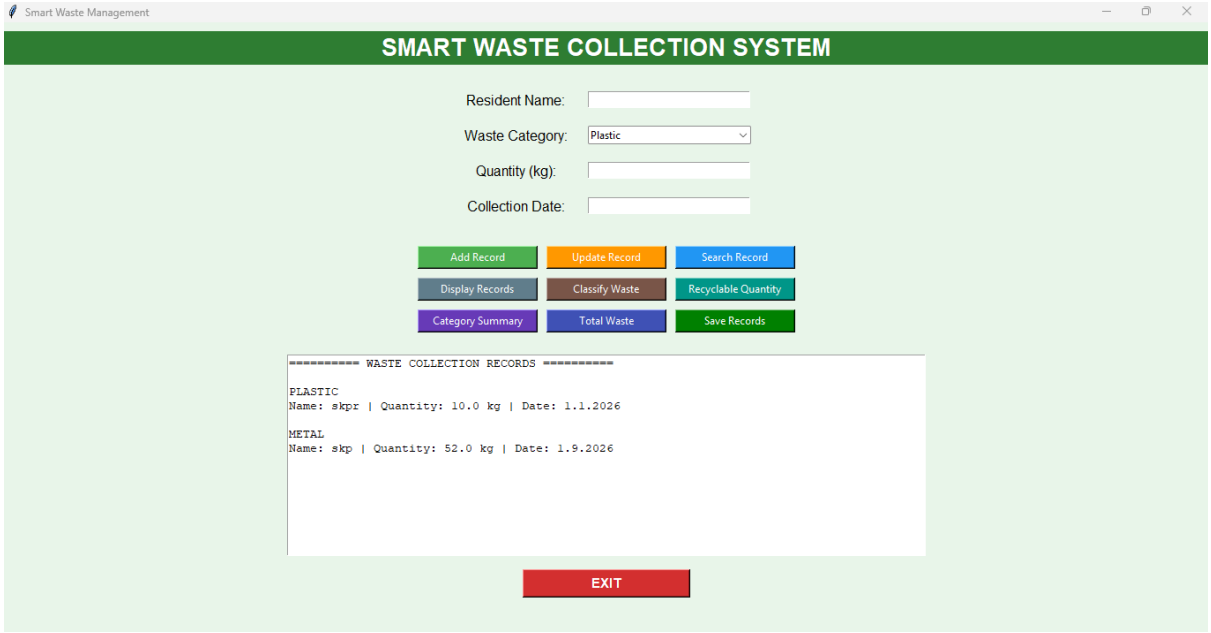
```
date_entry.grid(
    row=3,
    column=1,
    padx=10,
    pady=8
)

button_frame = tk.Frame(
    window,
    bg="#e8f5e9"
)

button_frame.pack(pady=10)

tk.Button(
    button_frame,
    text="Add Record",
    width=18,
    command=add_record,
    bg="#4CAF50",
    fg="white"
).grid(row=0, column=0, padx=5, pady=5)

tk.Button(
    button_frame,
    text="Update Record",
    width=18,
    command=update_record,
    bg="#FF9800",
    fg="white"
).grid(row=0, column=1, padx=5, pady=5)
```

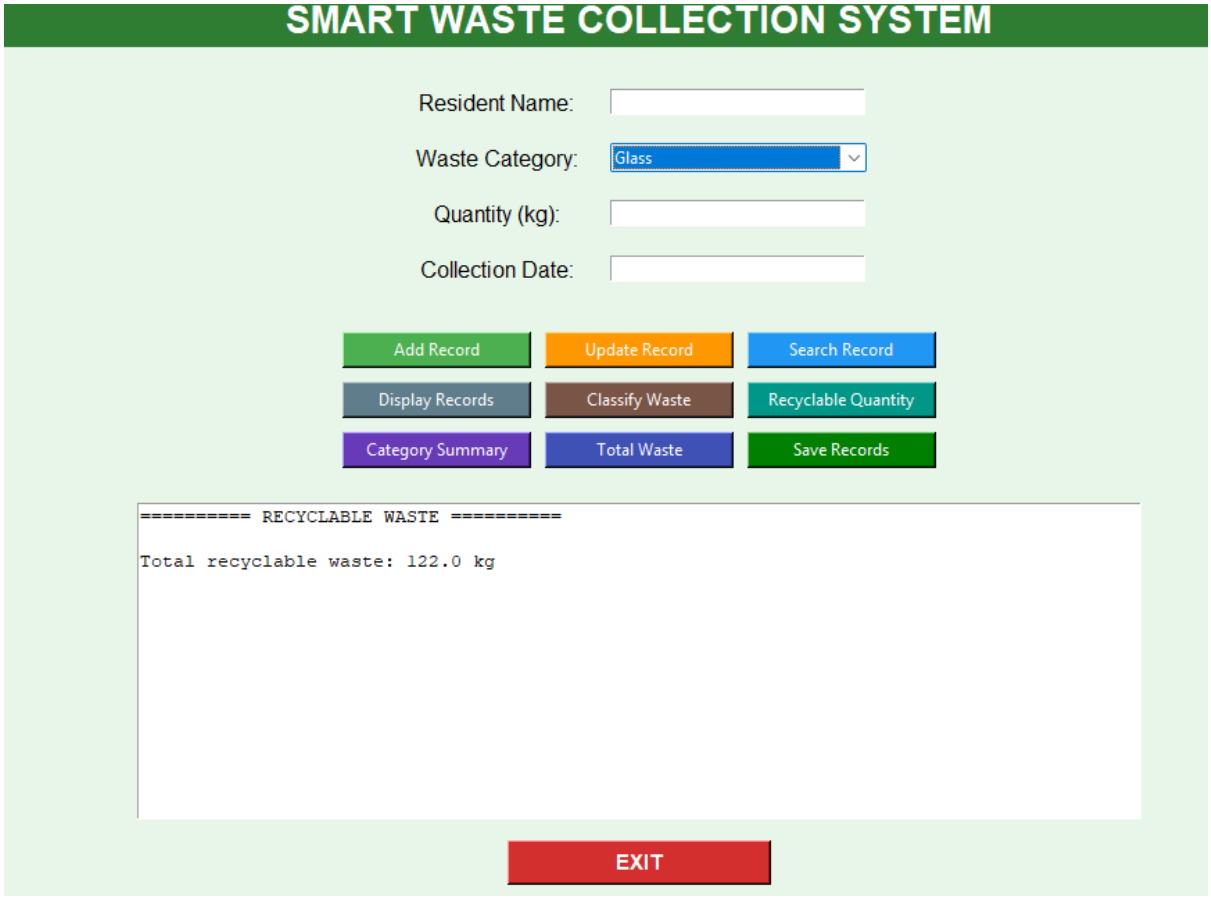
```
tk.Button(
    button_frame,
    text="Search Record",
    width=18,
    command=search_record,
    bg="#2196F3",
    fg="white"
).grid(row=0, column=2, padx=5, pady=5)

tk.Button(
    button_frame,
    text="Display Records",
    width=18,
    command=show_records,
    bg="#607D8B",
    fg="white"
).grid(row=1, column=0, padx=5, pady=5)

tk.Button(
    button_frame,
    text="Classify Waste",
    width=18,
    command=classify_waste,
    bg="#795548",
    fg="white"
).grid(row=1, column=1, padx=5, pady=5)

tk.Button(
    button_frame,
    text="Recyclable Quantity",
```

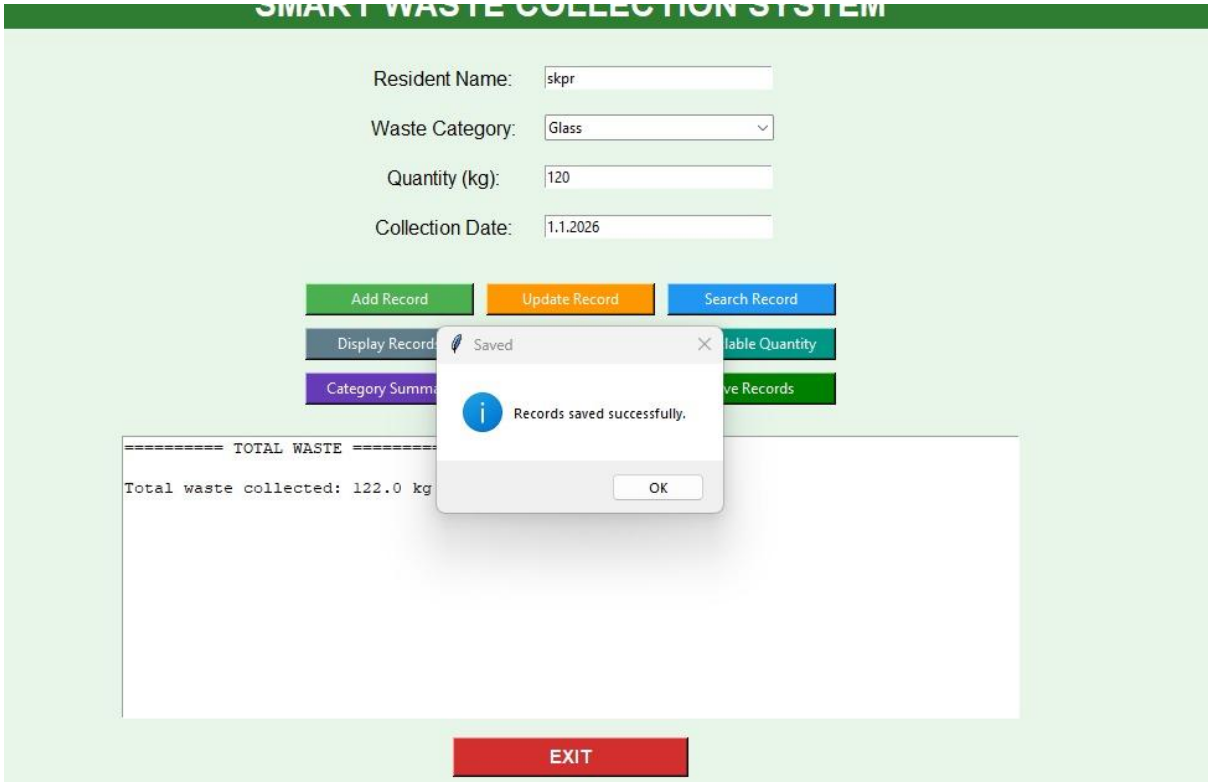
```
    width=18,
    command=recyclable_quantity,
    bg="#009688",
    fg="white"
).grid(row=1, column=2, padx=5, pady=5)

tk.Button(
    button_frame,
    text="Category Summary",
    width=18,
    command=category_summary,
    bg="#673AB7",
    fg="white"
).grid(row=2, column=0, padx=5, pady=5)

tk.Button(
    button_frame,
    text="Total Waste",
    width=18,
    command=total_waste,
    bg="#3F51B5",
    fg="white"
).grid(row=2, column=1, padx=5, pady=5)

tk.Button(
    button_frame,
    text="Save Records",
    width=18,
    command=save_records,
    bg="#008000",
    fg="white"
```

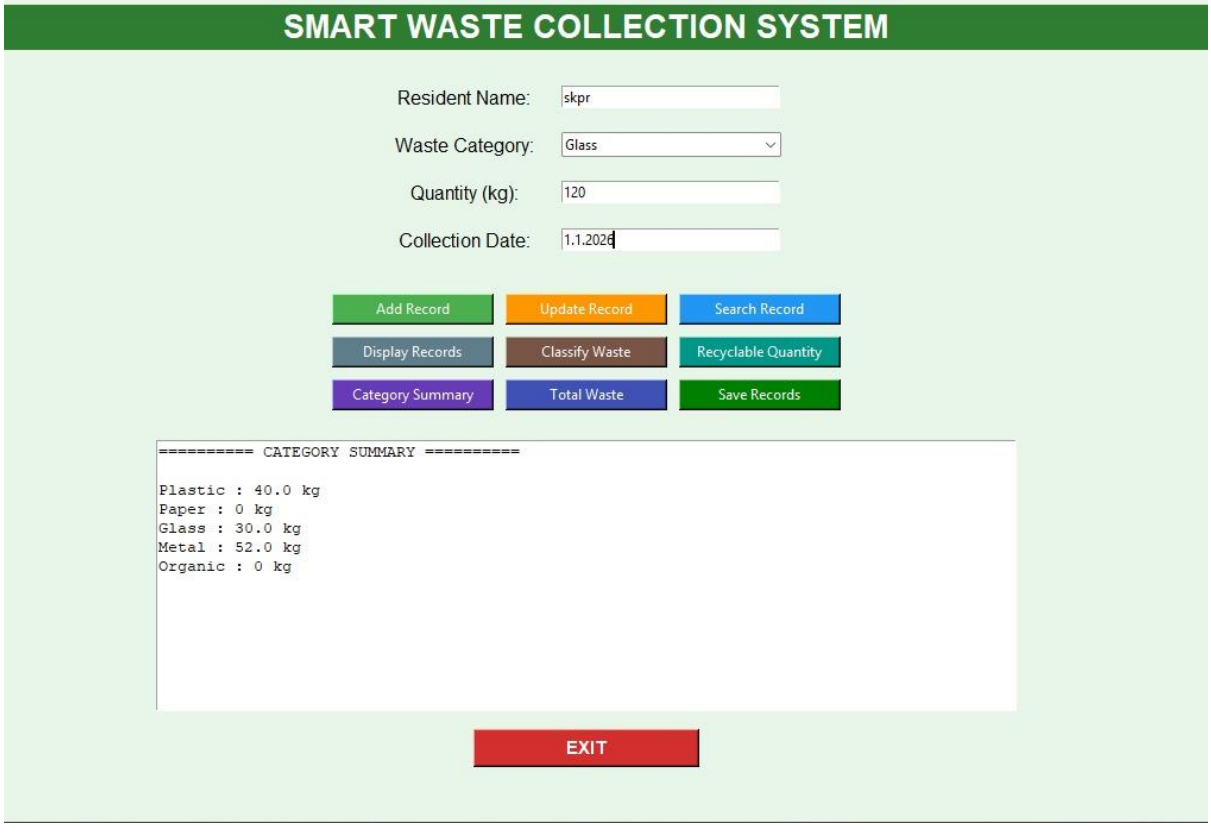
```
).grid(row=2, column=2, padx=5, pady=5)

output = tk.Text(
    window,
    width=90,
    height=14,
    font=("Courier New", 10)
)

output.pack(
    padx=20,
    pady=10
)

tk.Button(
    window,
    text="EXIT",
    width=20,
    command=exit_program,
    bg="#D32F2F",
    fg="white",
    font=("Arial", 11, "bold")
).pack(pady=5)

load_records()

window.mainloop()
```

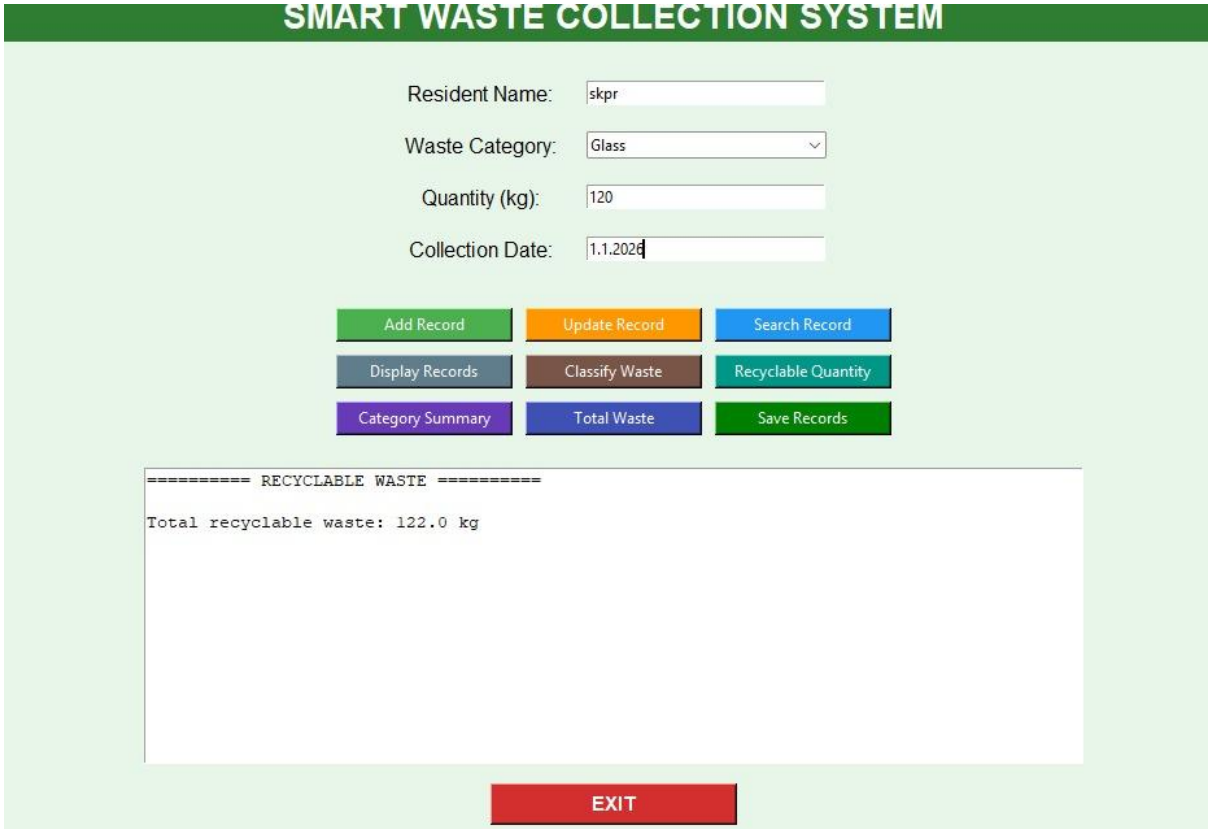
OUTPUTS

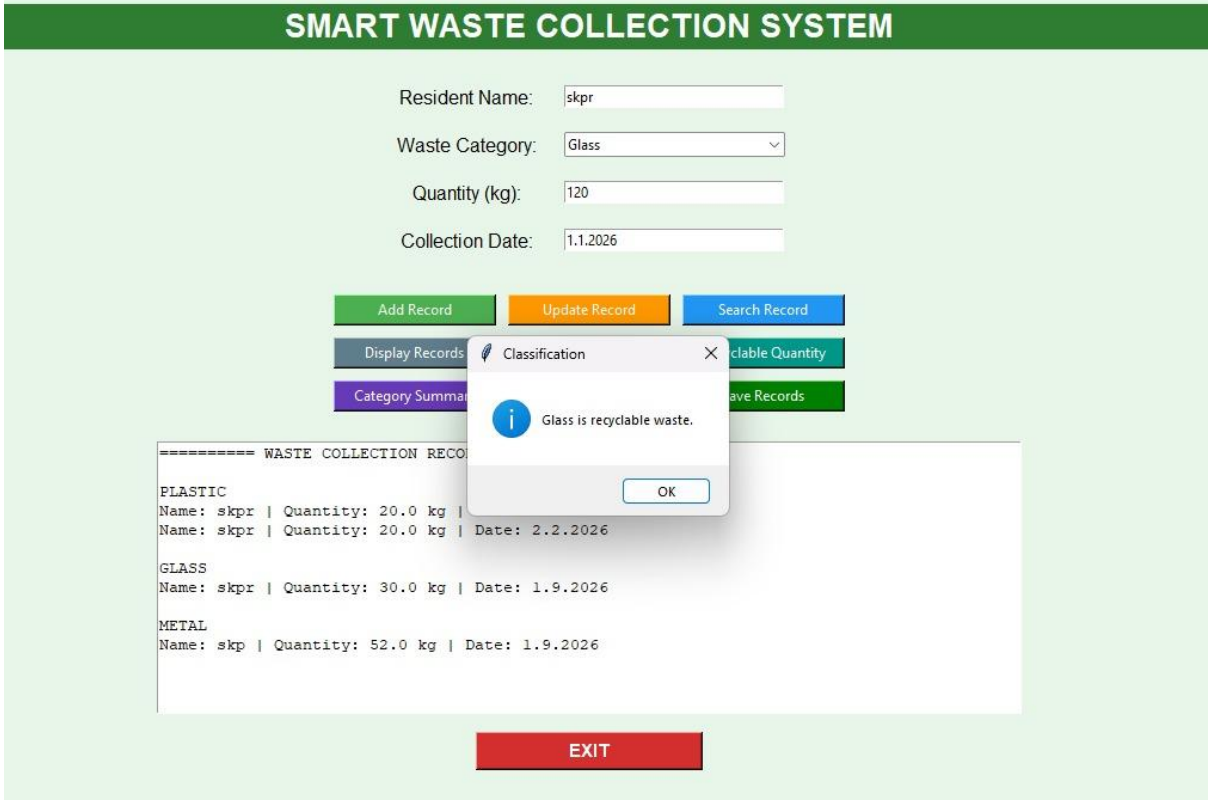

Smart Waste Management
SMART WASTE COLLECTION SYSTEM
Resident Name:
Waste Category:
Plastic
Quantity (kg):
Collection Date:
Add Record
Update Record
Search Record
Display Records
Classify Waste
Recyclable Quantity
Category Summary
Total Waste
Save Records
========== WASTE COLLECTION RECORDS ==========
PLASTIC
Name: skpr | Quantity: 10.0 kg | Date: 1.1.2026
METAL
Name: skp | Quantity: 52.0 kg | Date: 1.9.2026
EXIT

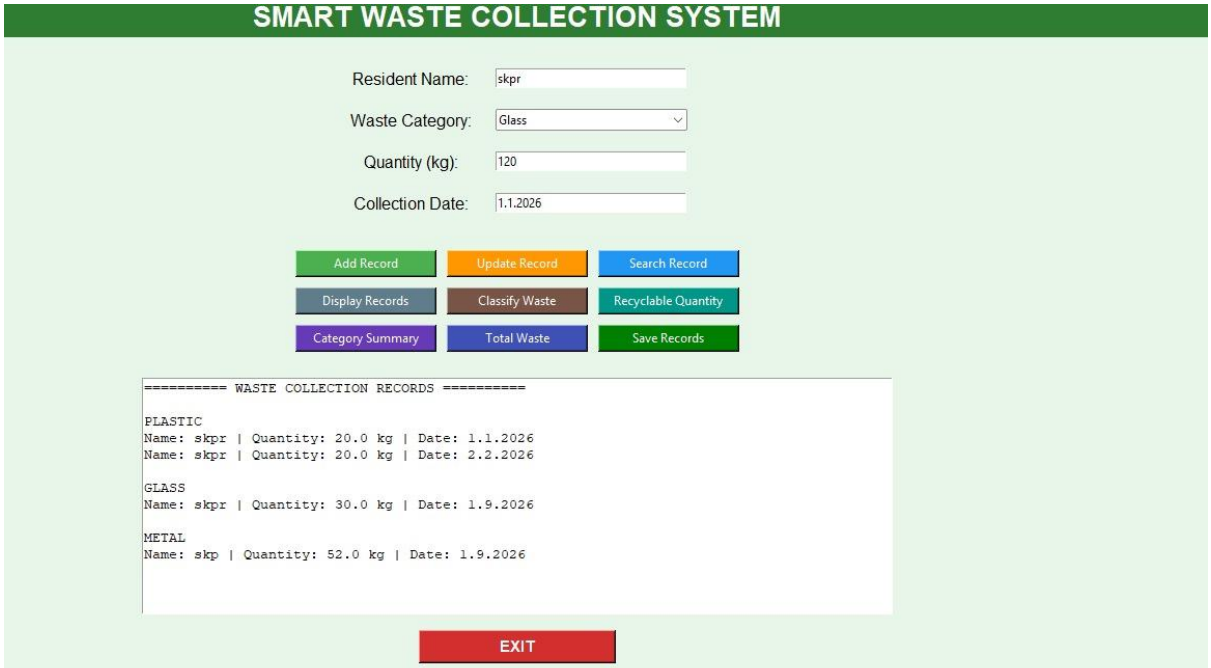

SMART WASTE COLLECTION SYSTEM
Resident Name:
Waste Category:
Glass
Quantity (kg):
Collection Date:
Add Record
Update Record
Search Record
Display Records
Classify Waste
Recyclable Quantity
Category Summary
Total Waste
Save Records
========== RECYCLABLE WASTE ==========
Total recyclable waste: 122.0 kg
EXIT


Resident Name:
skpr
Waste Category:
Glass
Quantity (kg):
120
Collection Date:
1.1.2026
Add Record
Update Record
Search Record
Display Records
Saved
Records saved successfully.
OK
Category Summary
Records
========== TOTAL WASTE ==========
Total waste collected: 122.0 kg
EXIT


SMART WASTE COLLECTION SYSTEM
Resident Name:
skpr
Waste Category:
Glass
Quantity (kg):
120
Collection Date:
1.1.2026
Add Record
Update Record
Search Record
Display Records
Classify Waste
Recyclable Quantity
Category Summary
Total Waste
Save Records
========== CATEGORY SUMMARY ==========
Plastic : 40.0 kg
Paper : 0 kg
Glass : 30.0 kg
Metal : 52.0 kg
Organic : 0 kg
EXIT


SMART WASTE COLLECTION SYSTEM
Resident Name:
skpr
Waste Category:
Glass
Quantity (kg):
120
Collection Date:
1.1.2026
Add Record
Update Record
Search Record
Display Records
Classify Waste
Recyclable Quantity
Category Summary
Total Waste
Save Records
========== RECYCLABLE WASTE ==========
Total recyclable waste: 122.0 kg
EXIT


SMART WASTE COLLECTION SYSTEM
Resident Name:
skpr
Waste Category:
Glass
Quantity (kg):
120
Collection Date:
1.1.2026
Add Record
Update Record
Search Record
Display Records
Category Summa
Classification
Glass is recyclable waste.
OK
========== WASTE COLLECTION RECO
PLASTIC
Name: skpr | Quantity: 20.0 kg |
Name: skpr | Quantity: 20.0 kg | Date: 2.2.2026
GLASS
Name: skpr | Quantity: 30.0 kg | Date: 1.9.2026
METAL
Name: skp | Quantity: 52.0 kg | Date: 1.9.2026
EXIT


SMART WASTE COLLECTION SYSTEM
Resident Name:
skpr
Waste Category:
Glass
Quantity (kg):
120
Collection Date:
1.1.2026
Add Record
Update Record
Search Record
Display Records
Classify Waste
Recyclable Quantity
Category Summary
Total Waste
Save Records
========== WASTE COLLECTION RECORDS ==========
PLASTIC
Name: skpr | Quantity: 20.0 kg | Date: 1.1.2026
Name: skpr | Quantity: 20.0 kg | Date: 2.2.2026
GLASS
Name: skpr | Quantity: 30.0 kg | Date: 1.9.2026
METAL
Name: skp | Quantity: 52.0 kg | Date: 1.9.2026
EXIT


SMART WASTE COLLECTION SYSTEM
Resident Name:
skpr
Waste Category:
Glass
Quantity (kg):
120
Collection Date:
1.1.2026
Add Record
Update Record
Search Record
Display Records
Classify Waste
Recyclable Quantity
Category Summary
Total Waste
Save Records
========== SEARCH RESULT ==========
Name: skpr
Category: Plastic
Quantity: 20.0 kg
Date: 1.1.2026
Name: skpr
Category: Plastic
Quantity: 20.0 kg
Date: 2.2.2026
Name: skpr
Category: Glass
EXIT

SMART WASTE COLLECTION SYSTEM

Resident Name:

Waste Category: Glass

Quantity (kg):

Collection Date:

Add Record | Update Record | Search Record

Display Records | Classify Waste | Recyclable Quantity

Category Summary | Total Waste | Save Records

```
========== WASTE COLLECTION RECORDS ==========

PLASTIC
Name: skpr | Quantity: 20.0 kg | Date: 1.1.2026
Name: skpr | Quantity: 20.0 kg | Date: 2.2.2026

GLASS
Name: skpr | Quantity: 30.0 kg | Date: 1.9.2026

METAL
Name: skp | Quantity: 52.0 kg | Date: 1.9.2026
```

EXIT